WEEK 6: ACID Transactions & the Lakehouse

From PostgreSQL Guarantees to Spark on Databricks

Why Transactions Matter

From PostgreSQL docs (§3.4):

> "The essential point of a transaction is that it bundles multiple steps into a single, all-or-nothing operation. The intermediate states between the steps are not visible to other concurrent transactions, and if some failure occurs that prevents the transaction from completing, then none of the steps affect the database at all."

Bank transfer — the full picture (PG tutorial example):

```
UPDATE accounts SET balance = balance - 100.00 WHERE name = 'Alice';
UPDATE branches SET balance = balance - 100.00
    WHERE name = (SELECT branch_name FROM accounts WHERE name = 'Alice');
UPDATE accounts SET balance = balance + 100.00 WHERE name = 'Bob';
UPDATE branches SET balance = balance + 100.00
    WHERE name = (SELECT branch_name FROM accounts WHERE name = 'Bob');
```

Four separate writes. Either all take effect or none should.

Atomicity and Durability — PG Official Language

From PostgreSQL docs:

> "A transaction is said to be atomic: from the point of view of other transactions, it either happens completely or not at all."
> "A transactional database guarantees that all the updates made by a transaction are logged in permanent storage (i.e., on disk) before the transaction is reported complete."
> "The updates made so far by an open transaction are invisible to other transactions until the transaction completes, whereupon all the updates become visible simultaneously."

Transaction block syntax:

```
BEGIN;
UPDATE accounts SET balance = balance - 100.00 WHERE name = 'Alice';
-- ... more updates ...
COMMIT;   -- or ROLLBACK to cancel everything
```

ACID — PostgreSQL Mapping

Property	PostgreSQL Behaviour
Atomicity	All steps complete or none. `ROLLBACK` cancels.
Consistency	Constraints enforced at commit; invalid state rejected.
Isolation	Open transaction changes invisible to other sessions.
Durability	Commits written to WAL on permanent storage before ack.

PostgreSQL note from docs:

> "PostgreSQL actually treats every SQL statement as being executed within a transaction. If you do not issue a BEGIN command, then each individual statement has an implicit BEGIN and (if successful) COMMIT wrapped around it."

Consistency via Constraints

Consistency: every transaction moves the database from one valid state to another.

```
CREATE TABLE departments (
  dept_id INT PRIMARY KEY,
  name    TEXT NOT NULL UNIQUE
);
CREATE TABLE employees (
  emp_id   INT PRIMARY KEY,
  name     TEXT NOT NULL,
  dept_id  INT NOT NULL REFERENCES departments(dept_id),
  salary   NUMERIC(12,2) CHECK (salary >= 0)
);
```

Initial data:

```
departments              employees
 dept_id | name          emp_id | name   | dept_id | salary
---------+-------------  -------+--------+---------+-----------
 10      | Engineering         1 | Alice  |      10 | 120000.00
 20      | Sales               2 | Bob    |      20 |  90000.00
```

Any attempt to insert a non-existent `dept_id`, a `NULL` name, or a negative salary fails — the database never lands in an invalid state.

The Four Concurrency Phenomena

Phenomenon	Definition
Dirty read	Transaction reads data written by a concurrent *uncommitted* transaction.
Nonrepeatable read	Transaction re-reads data and finds it changed by another *committed* transaction.
Phantom read	Transaction re-runs a query and finds the row set changed by another *committed* transaction.
Serialization anomaly	Result of committed group is inconsistent with any serial ordering of those transactions.

These are the formal problems that isolation levels exist to prevent.

Isolation Levels — Official Behaviour Table

From PostgreSQL §13.2 (Table 13.1):

Isolation Level	Dirty Read	Nonrepeatable Read	Phantom Read	Serialization Anomaly
Read Uncommitted	Allowed *(not in PG)*	Possible	Possible	Possible
Read Committed *(PG default)*	Not possible	Possible	Possible	Possible
Repeatable Read	Not possible	Not possible	Allowed *(not in PG)*	Possible
Serializable	Not possible	Not possible	Not possible	Not possible

Key PG note: "PostgreSQL's Read Uncommitted mode behaves like Read Committed." Only three distinct behaviours are implemented.

Read Committed — How It Works

From PG docs §13.2.1:

> "A SELECT query sees only data committed before the query began; it never sees either uncommitted data or changes committed by concurrent transactions during the query's execution."
> "Two successive SELECT commands can see different data, even though they are within a single transaction, if other transactions commit changes after the first SELECT starts and before the second SELECT starts."

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  -- the default
SELECT quantity FROM iso_inventory WHERE product_id = 1;
-- Meanwhile: another session commits UPDATE ... SET quantity = 8
SELECT quantity FROM iso_inventory WHERE product_id = 1;
COMMIT;
```

Session A — first read:

```
             note              | quantity
-------------------------------+----------
 RC: first read (expect 10) |       10
(1 row)
```

Session B commits (concurrently):

```
BEGIN;
UPDATE iso_inventory SET quantity = 8 WHERE product_id = 1;
COMMIT;
```

Session A — second read (same transaction, fresh snapshot):

```
                            note                             | quantity
-------------------------------------------------------------+----------
 RC: second read (expect 8 after client 2 committed) |        8
(1 row)
```

Each SELECT takes a fresh snapshot. Safe for simple updates; complex queries may see inconsistency.

Repeatable Read — Snapshot Isolation

From PG docs §13.2.2:

> "The Repeatable Read isolation level only sees data committed before the transaction began; successive SELECT commands within a single transaction see the same data, i.e., they do not see changes made by other transactions that committed after their own transaction started."

```
BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT quantity FROM iso_inventory WHERE product_id = 1;
-- Another session commits an update to quantity = 8
SELECT quantity FROM iso_inventory WHERE product_id = 1;
COMMIT;
```

Session A — first read (snapshot taken at BEGIN):

```
             note              | quantity
-------------------------------+----------
 RR: first read (expect 10) |       10
(1 row)
```

Session B commits concurrently:

```
BEGIN;
UPDATE iso_inventory SET quantity = 8 WHERE product_id = 1;
COMMIT;
```

Session A — second read (snapshot unchanged):

```
                            note                            | quantity
------------------------------------------------------------+----------
 RR: second read (still expect 10, snapshot is fixed) |       10
(1 row)
```

After Session A commits — fresh statement sees latest committed value:

```
                             note                             | quantity
--------------------------------------------------------------+----------
 After RR commit, fresh statement sees latest (expect 8) |        8
(1 row)
```

Write conflict error from PG docs:

```
ERROR:  could not serialize access due to concurrent update
```

Applications must be prepared to retry transactions that receive this error.

Serializable — Strictest Level

From PG docs §13.2.3:

> "The Serializable isolation level provides the strictest transaction isolation. This level emulates serial transaction execution for all committed transactions; as if transactions had been executed one after another, serially, rather than concurrently."

Official PG serialization anomaly example (`mytab`):

class	value
1	10
1	20
2	100
2	200

- Session A: `SELECT SUM(value) WHERE class = 1` → 30, inserts as `class = 2`
- Session B: `SELECT SUM(value) WHERE class = 2` → 300, inserts as `class = 1`
- Neither serial order produces these results → one transaction is rolled back:

```
ERROR:  could not serialize access due to read/write dependencies among transactions
```

Serialization failures always return SQLSTATE `'40001'`. Applications **must retry**.

Savepoints — Partial Rollback

From PostgreSQL docs (§3.4), exact example:

```
BEGIN;
UPDATE accounts SET balance = balance - 100.00
    WHERE name = 'Alice';
SAVEPOINT my_savepoint;
UPDATE accounts SET balance = balance + 100.00
    WHERE name = 'Bob';
-- oops, wrong account
ROLLBACK TO my_savepoint;
UPDATE accounts SET balance = balance + 100.00
    WHERE name = 'Wally';
COMMIT;
```

From docs:

> "ROLLBACK TO is the only way to regain control of a transaction block that was put in aborted state by the system due to an error, short of rolling it back completely and starting again."

Alice is debited. Bob's credit is undone. Wally is credited. Everything commits as one unit.

Final state (`SELECT name, balance FROM accounts ORDER BY name`):

```
 name  | balance
-------+---------
 Alice |  900.00
 Bob   |  500.00
 Wally |  600.00
(3 rows)
```

Durability and WAL

From PostgreSQL WAL docs:

- Write-Ahead Logging (WAL) is a standard method for ensuring data integrity.
- All changes to data files are written to the WAL log *before* they are written to the data files themselves.
- After a crash, WAL records are replayed to bring the database to a consistent state.

> "A transactional database guarantees that all the updates made by a transaction are logged in permanent storage (i.e., on disk) before the transaction is reported complete."

What this means in practice:

- A confirmed `COMMIT` will survive a crash.
- Uncommitted changes are never applied to durable storage.
- Recovery replays WAL from last checkpoint forward.

Two-Client Isolation Lab

Run with two open terminal sessions to observe isolation behavior directly.

Files:

- `code/week6_client1.sql` — Session A (opens transactions, waits, re-reads)
- `code/week6_client2.sql` — Session B (commits an update while A waits)

Script flow:

1. Client 1 opens `READ COMMITTED` txn, reads `quantity = 10`
2. Client 2 updates `quantity = 8`, commits
3. Client 1 reads again — sees `8` *(RC behavior)*
4. Reset; repeat with `REPEATABLE READ`
5. Client 1 reads again — still sees `10` *(RR snapshot behavior)*
   This directly demonstrates the phenomena definitions from PG docs.

Stored Procedure Pattern (PostgreSQL)

Business logic wrapped in a function inherits the surrounding transaction context.

```
CREATE OR REPLACE FUNCTION create_order(
  p_customer_name TEXT, p_product_id INT, p_quantity INT
) RETURNS INT LANGUAGE plpgsql AS $$
DECLARE
  v_order_id INT; v_price NUMERIC(12,2); v_available INT;
BEGIN
  SELECT price, quantity INTO v_price, v_available
  FROM products WHERE product_id = p_product_id;
  IF NOT FOUND THEN RAISE EXCEPTION 'Product % not found', p_product_id; END IF;
  IF p_quantity <= 0 THEN RAISE EXCEPTION 'Quantity must be positive'; END IF;
  IF v_available < p_quantity THEN RAISE EXCEPTION 'Insufficient stock: available %, requested %', v_available, p_quantity; END IF;
```

```
  INSERT INTO orders(customer_name, order_date, total)
  VALUES (p_customer_name, CURRENT_DATE, v_price * p_quantity)
  RETURNING order_id INTO v_order_id;
  INSERT INTO order_items(order_id, product_id, quantity, price)
  VALUES (v_order_id, p_product_id, p_quantity, v_price);
  UPDATE products SET quantity = quantity - p_quantity
  WHERE product_id = p_product_id;
  RETURN v_order_id;
END;
$$;
```

Why Spark Exists

From Databricks docs:

> "Apache Spark is the technology powering compute clusters and SQL warehouses in Databricks."

One machine and one database are not enough for:

- Petabyte-scale data
- Sub-second streaming ingestion
- Distributed machine learning

What Databricks Looks Like

Databricks is a managed cloud platform (the "lakehouse") on AWS / Azure / GCP — all in the browser, nothing to install. It packages this week's pieces into one product:

- **Spark** — compute engine (clusters / serverless)
- **Delta Lake** — default ACID table format (time travel, history)
- **Notebooks, SQL editor, dashboards, jobs, ML** on top

The left sidebar groups your tools:

Area	What it holds
Workspace	Notebooks, folders, Git repos
Catalog	Unity Catalog: catalogs → schemas → tables & volumes
Compute	Clusters / serverless to attach notebooks to
SQL	SQL editor, queries, dashboards, warehouses
Jobs / Pipelines	Scheduling & orchestration (Lakeflow)

A notebook attaches to **compute**, then runs Python / SQL / Scala / R cells against it.

Sign Up (Free Edition) & Connect

Databricks Free Edition — the no-cost tier for learners; it replaced Community Edition (retired 1 Jan 2026). Serverless and quota-limited, with **no cloud account or credit card** required.

1. Go to **databricks.com/learn/free-edition** (or `login.databricks.com`)
2. Sign up with Google, Microsoft, or email
3. A serverless **workspace** is created automatically — you land straight in the UI

Connect two ways:

- **Browser** — create a notebook in *Workspace*, attach compute, run cells.
- **VS Code** — install the **Databricks extension**, authenticate, then enable **Databricks Connect** (Runtime 13.3+): cells run locally while DataFrame operations execute on remote Databricks compute.

> Our Week 6 notebook runs on Databricks unchanged — just **drop the SparkSession setup cell** (`spark` is provided) and use a workspace / Unity Catalog path instead of the local Docker path. Step-by-step: `environment/databricks/` .

Local PySpark + Delta Setup

For this lab, Spark runs locally in Docker — no Databricks account needed.

```
cd environment/pyspark/pyspark_docker
docker compose up -d    # first run builds image (~5 min)
```

Connect VS Code to the Jupyter kernel:

1. Command Palette → **Jupyter: Specify Jupyter Server for Connections**
2. Enter: `http://localhost:8888/?token=pyspark`

SparkSession with Delta configured in the notebook:

```
from delta import configure_spark_with_delta_pip
spark = configure_spark_with_delta_pip(
    SparkSession.builder.appName("Week6Lab")
).getOrCreate()
```

Spark's core model:

1. **Partition** data across many nodes
2. **Transform** each partition in parallel
3. **Aggregate** results back

From Databricks DataFrame tutorial:

> "Apache Spark DataFrames provide a rich set of functions (select columns, filter, join, aggregate) that allow you to solve common data analysis problems efficiently."
> "Spark DataFrames and Spark SQL use a unified planning and optimization engine, allowing you to get nearly identical performance across all supported languages on Databricks (Python, SQL, Scala, and R)."

Spark DataFrame Core Operations

```
from pyspark.sql import functions as F
data = [
    (1, "C01", "Laptop",   120.50),
    (2, "C02", "Mouse",     25.00),
    (3, "C01", "Keyboard",  70.00),
    (4, "C03", "Monitor",  220.00),
]
df = spark.createDataFrame(data, ["order_id", "customer_id", "product", "amount"])
df.show()
```

```
+--------+-----------+--------+------+
|order_id|customer_id| product|amount|
+--------+-----------+--------+------+
|       1|        C01|  Laptop| 120.5|
|       2|        C02|   Mouse|  25.0|
|       3|        C01|Keyboard|  70.0|
|       4|        C03| Monitor| 220.0|
+--------+-----------+--------+------+
```

```
# filter
df.filter(F.col("amount") > 50).show()
# select + orderBy
from pyspark.sql.functions import desc
df.select("product", "amount").orderBy(desc("amount")).show()
# groupBy + agg (orderBy so the output is deterministic)
(df.groupBy("customer_id").agg(F.sum("amount").alias("total_spend"))
   .orderBy(F.col("total_spend").desc()).show())
```

```
# filter — 3 rows where amount > 50
+--------+-----------+--------+------+
|order_id|customer_id| product|amount|
+--------+-----------+--------+------+
|       1|        C01|  Laptop| 120.5|
|       3|        C01|Keyboard|  70.0|
|       4|        C03| Monitor| 220.0|
+--------+-----------+--------+------+
```

```
# groupBy + agg (ordered by total_spend desc)
+-----------+-----------+
|customer_id|total_spend|
+-----------+-----------+
|        C03|      220.0|
|        C01|      190.5|
|        C02|       25.0|
+-----------+-----------+
```

Transformations vs actions:

- Transformations (`.filter` , `.select` , `.groupBy`) build a *lazy* query plan.
- Actions (`.show()` , `.count()` , `.write`) trigger execution.

Databricks ACID — How Delta Implements It

From Databricks ACID docs:

> "Databricks uses Delta Lake by default for all reads and writes and builds upon the ACID guarantees provided by the open source Delta Lake protocol."

How each property is implemented:

Property	Databricks mechanism
Atomicity	Transaction log: data files written first; a new log entry becomes the commit. If a transaction fails, untracked files are cleaned up by `VACUUM`.
Consistency	Schema enforcement and table constraints are validated on write; an invalid write is rejected, never partially applied.
Isolation	Optimistic concurrency (no locks): Read → Write → *Validate & commit*; conflicts raise `ConcurrentModificationException`. Write-serializable for writes, snapshot isolation for reads; deadlock is not possible.
Durability	Data files and transaction logs live in cloud object storage (high availability + durability).

From docs: "For managing concurrent transactions, Databricks uses optimistic concurrency control. This means that there are no locks on reading or writing against a table, and deadlock is not a possibility."

Medallion Architecture

From Databricks docs:

> "A medallion architecture is a data design pattern used to organize data logically. Its goal is to incrementally and progressively improve the structure and quality of data as it flows through each layer of the architecture (from Bronze → Silver → Gold layer tables)."

Layer	What happens	Typical users
Bronze	Raw ingestion. Minimal validation. Store as-is. Preserve fidelity.	Data engineers, compliance/audit
Silver	Cleansing, deduplication, type casting, joins, schema enforcement.	Data engineers, analysts, data scientists
Gold	Dimensional modelling, aggregation, business-aligned datasets.	Business analysts, BI developers, ML engineers, executives

Data flowing through layers:

```
bronze.orders_raw          →  silver.orders_clean      →  gold.daily_sales
 order_ts = "2026/05/10"       order_ts = 2026-05-10       date       | revenue
 amount   = "120.50"           amount   = 120.50(DECIMAL)  2026-05-10 | 120.50
```

Medallion in the notebook (local PySpark):

```
DELTA_BASE = "/home/jovyan/work/delta"
# Bronze: raw records
df.write.format("delta").mode("overwrite").save(f"{DELTA_BASE}/bronze_orders")
# Silver: cast amount to DECIMAL, deduplicate
silver_df = (spark.read.format("delta").load(f"{DELTA_BASE}/bronze_orders")
             .withColumn("amount", F.col("amount").cast("decimal(12,2)"))
             .dropDuplicates(["order_id"]))
silver_df.write.format("delta").mode("overwrite").save(f"{DELTA_BASE}/silver_orders")
# Gold: aggregate revenue per customer
gold_df = (spark.read.format("delta").load(f"{DELTA_BASE}/silver_orders").groupBy("customer_id").agg(F.sum("amount").alias("total_revenue")))
gold_df.write.format("delta").mode("overwrite").save(f"{DELTA_BASE}/gold_customer_revenue")
spark.read.format("delta").load(f"{DELTA_BASE}/gold_customer_revenue").orderBy("customer_id").show()
```

```
+-----------+-------------+
|customer_id|total_revenue|
+-----------+-------------+
|        C01|       190.50|
|        C02|        25.00|
|        C03|       220.00|
+-----------+-------------+
```

Delta Lake — Key Properties

From Databricks docs:

> "Delta Lake is open source software that extends Parquet data files with a file-based transaction log for ACID transactions and scalable metadata handling."

> "All tables on Databricks are Delta tables by default."

Basic Delta table lifecycle (local PySpark):

```
DELTA_BASE = "/home/jovyan/work/delta"

# silver_orders already exists from the Medallion step above:
# version 0 = the cast (DECIMAL) + deduplicated write.

# Update (version 1)
from delta.tables import DeltaTable
dt = DeltaTable.forPath(spark, f"{DELTA_BASE}/silver_orders")
dt.update(condition="order_id = 1", set={"amount": F.lit(121.50)})

# History
dt.history().select("version", "timestamp", "operation", "operationMetrics").show(truncate=False)
```

Time Travel and Table History

From Databricks docs:

> "Each operation that modifies a table creates a new table version. Use history information to audit operations, rollback a table, or query a table at a specific point in time using time travel."

DESCRIBE HISTORY output (schema):

Column	Meaning
version	Table version generated by the operation
timestamp	When this version was committed
operation	Name of the operation (WRITE, UPDATE, DELETE, ...)
operationMetrics	Row/file counts for the operation

Time travel (local PySpark):

```
# Read version 0 — amount for order_id=1 is still 120.50
spark.read.format("delta")
.option("versionAsOf", 0)
.load(f"{DELTA_BASE}/silver_orders")
.orderBy("order_id").show()
```

```
+--------+-----------+--------+------+
|order_id|customer_id| product|amount|
+--------+-----------+--------+------+
|       1|        C01|  Laptop|120.50|
|       2|        C02|   Mouse| 25.00|
|       3|        C01|Keyboard| 70.00|
|       4|        C03| Monitor|220.00|
+--------+-----------+--------+------+
```

```
# Current version — order_id=1 updated to 121.50
spark.read.format("delta").load(f"{DELTA_BASE}/silver_orders").orderBy("order_id").show()
```

```
+--------+-----------+--------+------+
|order_id|customer_id| product|amount|
+--------+-----------+--------+------+
|       1|        C01|  Laptop|121.50|
|       2|        C02|   Mouse| 25.00|
|       3|        C01|Keyboard| 70.00|
|       4|        C03| Monitor|220.00|
+--------+-----------+--------+------+
```

Retention: history kept 30 days by default (`delta.logRetentionDuration`). Data files kept 7 days (`delta.deletedFileRetentionDuration`).

Summary

Concept	Key fact
Transactions	Atomic, invisible until commit, logged for durability
Implicit transactions	Every SQL statement has implicit BEGIN/COMMIT
Isolation (PG)	RC default; RR snapshot at txn start; SERIAL emulates serial order
Anomalies	Dirty read / nonrepeatable read / phantom read / serialization anomaly
Savepoints	ROLLBACK TO keeps earlier work, discards later work
WAL	Changes logged before data files; enables crash recovery

Databricks/Spark side:

Concept	Key fact
Spark	Powers compute clusters and SQL warehouses on Databricks
DataFrames	2D labeled structures; lazy transformations + eager actions
Delta ACID	Optimistic concurrency; write-serializable + snapshot reads; no locks
Medallion	Bronze (raw) → Silver (clean) → Gold (aggregated)
Time travel	VERSION AS OF / TIMESTAMP AS OF; DESCRIBE HISTORY

EXERCISES

Exercise 1: Transaction Block and ROLLBACK

```sql
BEGIN;
UPDATE products SET quantity = quantity - 2 WHERE product_id = 1;
INSERT INTO orders(customer_name, order_date, total)
VALUES ('Alice', CURRENT_DATE, 1999.98);
-- force a constraint violation
INSERT INTO order_items(order_id, product_id, quantity, price)
VALUES (99999, -1, 2, 999.99);
COMMIT;
```

1. What is the state of `products.quantity` after the error?
2. According to PG docs, why is atomicity the right guarantee here?

Exercise 2: Two-Client Isolation Lab

Using `code/week6_client1.sql` (Session A) and `code/week6_client2.sql` (Session B):

1. Run under `READ COMMITTED`. What does Session A's second read return?
2. Run under `REPEATABLE READ`. What does Session A's second read return now?
3. Which phenomenon from the official PG table explains the difference?

Exercise 3: Savepoint Lab

Using the exact savepoint example from PG docs §3.4:

```
BEGIN;
UPDATE accounts SET balance = balance - 100.00 WHERE name = 'Alice';
SAVEPOINT my_savepoint;
UPDATE accounts SET balance = balance + 100.00 WHERE name = 'Bob';
ROLLBACK TO my_savepoint;
UPDATE accounts SET balance = balance + 100.00 WHERE name = 'Wally';
COMMIT;
```

1. Which account balances changed?
2. Why is `ROLLBACK TO my_savepoint` the only way to recover from an error inside a transaction block without aborting the whole transaction?

Exercise 4: Serializable Anomaly

Recreate the `mytab` example from PG docs §13.2.3 in two sessions:

```
CREATE TABLE mytab (class INT, value INT);
INSERT INTO mytab VALUES (1,10),(1,20),(2,100),(2,200);
-- Session A (SERIALIZABLE)
BEGIN; SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT SUM(value) FROM mytab WHERE class = 1;  -- 30
INSERT INTO mytab VALUES (2, 30);
COMMIT;
```

```
-- Session B (SERIALIZABLE, concurrent with A)
BEGIN; SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT SUM(value) FROM mytab WHERE class = 2;  -- 300
INSERT INTO mytab VALUES (1, 300);
COMMIT;
```

1. Which session receives the serialization failure?
2. What SQLSTATE code does PG return on serialization failure, and what should the application do?

Exercise 5: PySpark Notebook Lab

Open `code/week6_databricks_notebook.ipynb` (connect VS Code to `http://localhost:8888/?token=pyspark` — see `environment/pyspark/readme.md`):

1. Run the SparkSession setup cell and confirm `Spark 3.5.x — Delta ready`.
2. Run `.filter()` and `.groupBy().agg()` — verify the output matches the expected tables.
3. Run the Medallion section — confirm Bronze, Silver, Gold Delta tables are written.
4. Run `dt.history()` — what does version 0 show for `operation` and `operationMetrics`?
5. Run time travel `VERSION AS OF 0` — confirm `amount = 120.50` for order_id 1.

Exercise 6: Run It on Databricks (Free Edition)

Using `code/week6_databricks_project/` (see its `readme.md` and `environment/databricks/`):

1. Sign up for **Databricks Free Edition** and import the project notebook.
2. List what you had to change vs the local notebook. *(Hint: the SparkSession setup cell and the storage path.)*
3. Run the medallion + time-travel cells. Confirm `VERSION AS OF 0` shows `amount = 120.50` for order_id 1 and the current version shows `121.50` — the **same result** as the local Docker run.
4. Run `DESCRIBE HISTORY` — which `operation` produced version 1, and what does `operationMetrics` report for `numUpdatedRows`?
5. Optimistic concurrency: if two notebooks updated the same Delta table at the same instant, what happens — and which ACID property covers it?

Exercise 7: Reflection

1. PostgreSQL says every statement is implicitly transactional. Why is that important?
2. Under `READ COMMITTED`, two SELECTs in the same transaction can return different values. Which phenomenon is that?
3. What error does PostgreSQL raise when a `REPEATABLE READ` transaction encounters a concurrent write, and what must the application do?
4. Databricks uses optimistic concurrency — no locks. What is the trade-off vs. pessimistic locking?
5. Why does the Medallion architecture separate Bronze from Silver rather than cleaning data on ingestion?